

个人作业报告

题目：**大数据原理与技术**

作业三：K-means 与 DBSCAN 算法

姓名	林宏宇
学号	23320093
院系	计算机学院
专业	计算机科学与技术
指导教师	王昌栋

2026 年 4 月 5 日

【摘　要】

在学习王老师的大数据原理与技术课程[1]后，在此报告中完成相应的课后作业。

本作业围绕 “聚类算法对比实验” 展开，要求使用 Python 完成**两部分任务**。**K-means 与 DBSCAN 聚类算法实现**：在鸢尾花数据集上实现 K-means 和 DBSCAN 算法，调整超参数（如簇数、邻域半径）观察聚类结果变化。**聚类性能评估**：用准确率、轮廓系数和 Calinski-Harabasz 指数评估性能。

作业提交要求：

- 提交内容：代码与作业报告（2 页以上）
- 命名格式：“学号 + 姓名 + 作业 3.zip”
- 提交至邮箱：BDSysu2026Spring@163.com
- 截止时间：布置作业之后的下一个周日 24:00
- 严禁抄袭：任何一次抄袭，平时成绩按零分处理

本次实验流程首先加载并分析鸢尾花数据集，对特征做 Z-score 标准化；其次手写实现 K-means 与 DBSCAN 并在参数网格上进行搜索（K-means 的 k，DBSCAN 的 ε 与 min_samples）；对每组结果计算 Accuracy（标签对齐）、Silhouette 与 Calinski–Harabasz 指数并绘制 PCA 二维投影进行可视化；最后汇总指标与图像，比较两算法性能并给出参数选择与复现建议。

实验结果表明，在鸢尾花数据集上，手写实现的 K-means 在 $k = 3$ 时能够较好恢复真实类别（Accuracy = 0.8533）；DBSCAN 在 $\varepsilon = 0.8$ 且 min_samples = 3 时取得较为平衡的结果（Accuracy 约 0.66，噪声点 4 个）。文中手工实现了两种算法并基于 Accuracy、Silhouette 与 Calinski–Harabasz 指数进行比较，结合参数扫描与可视化分析给出参数选择建议与复现性实践。

关键词： 聚类算法，K-means，DBSCAN，聚类性能评估，鸢尾花数据集

目录

1 聚类算法对比实验 1
1.1 实验目标 1
1.2 数据集与预处理 1
1.3 算法原理 2
1.4 核心算法伪代码 2
1.5 参数设置与实验设计 3
1.6 评估指标设计与实现 4
1.7 实验结果 4
1.8 实验结果分析与评价 7

2 总结 9
2.1 实验总结 9
2.2 技术学习总结 9
2.3 心得体会 9

参考文献 10

附录 A 实验代码 11
A.1 聚类算法代码清单 11
A.2 K-means 代码实现细节 11
A.3 DBSCAN 代码实现细节 13

1 聚类算法对比实验

1.1 实验目标

本章在鸢尾花（Iris）数据集上实现并比较两种常用的无监督聚类算法：手写实现的 **K-means** 与 **DBSCAN**。重点内容包括：

- 手写实现两种算法的核心逻辑（禁止直接使用 sklearn 的聚类实现）；
- 通过调整超参数（K-means 的 k，DBSCAN 的 ε 与 min_samples）观察聚类结果变化；
- 使用准确率（需对齐簇标签）、轮廓系数与 Calinski-Harabasz 指数对聚类性能进行评估；
- 将结果以表格与可视化图像展示，并在报告中分析和讨论。

1.2 数据集与预处理

本实验使用 sklearn 提供的鸢尾花（Iris）数据集。数据包含 $n = 150$ 个样本，每个样本有 $d = 4$ 个数值特征：萼片长度、萼片宽度、花瓣长度、花瓣宽度；并带有 3 类真实标签（Setosa、Versicolor、Virginica）。

预处理步骤如下：

1) 缺失值检查：原数据无缺失值，直接使用；
2) 特征标准化：对每一维特征做 Z-score 标准化，公式为

$$x' = \frac{x - \mu}{\sigma},$$

其中 μ、σ 分别为该维度的样本均值与标准差；标准化可以避免不同量纲对距离计算的影响；

1.3 算法原理

1.3.1 K-means 原理

K-means 试图将样本划分为 k 个簇，使得簇内方差最小化。定义簇标签 $z_i \in \{1, \ldots, k\}$，簇中心为 μ_j，优化目标为：

$$\min_{z,\{\mu_j\}} J = \sum_{i=1}^{n} ||x_i - \mu_{z_i}||^2.$$

给定簇分配时，最优质心为每簇点的均值：

$$\mu_j = \frac{1}{|C_j|} \sum_{i \in C_j} x_i.$$

交替优化（E-step：按最近中心分配；M-step：按簇均值更新中心）会使目标函数单调下降并在有限步内收敛到局部极小点。K-means 对初始中心敏感，常用多次随机初始化取最优解。

1.3.2 DBSCAN 原理

DBSCAN 基于密度的聚类思想：

- 对每个点 p，其 ε-邻域定义为 $N_\varepsilon(p) = \{q : ||q - p|| \le \varepsilon\}$；
- 若 $|N_\varepsilon(p)| \ge$ min_samples，则 p 为**核心点**；
- 核心点之间若可达（通过核心点链），则属于同一簇；非核心但位于某核心点邻域内的点为**边界点**；既非核心也不在任何核心邻域内的点为**噪声**。

DBSCAN 无需预先指定簇数，可发现任意形状簇并自动识别离群点，但对 ε 与 min_samples 敏感。

1.4 核心算法伪代码

1.4.1 K-means

K-means 的基本流程为：随机初始化 k 个质心，重复进行样本到质心的最近邻分配与质心重计算，直至收敛。

```
Input: X (n x d), k, max_iters, tol
Initialize centroids C by sampling k points from X
for t in range(max_iters):
```

```
    for each sample i: assign label[i] = argmin_j ||X[i]-C[j]||
    for each cluster j: C_new[j] = mean of members
    if ||C_new - C|| < tol: break
    C = C_new
return labels, C
```

1.4.2 DBSCAN

DBSCAN 通过密度连接定义簇：对于每个点，若其 ε 邻域内点数不小于 min_samples，则为核心点并扩展簇。

```
Input: X, eps, min_samples
labels = UNVISITED
cluster_id = 0
for each point p:
    if p is visited: continue
    N = neighbors(p, eps)
    if |N| < min_samples: labels[p] = NOISE
    else:
        labels[p] = cluster_id
        seeds = N \ {p}
        for q in seeds:
            if q is NOISE: labels[q] = cluster_id
            if q not visited:
                labels[q] = cluster_id
                Nq = neighbors(q, eps)
                if |Nq| >= min_samples: seeds += Nq
        cluster_id += 1
return labels
```

上述指标均在代码中手写实现，没有使用 sklearn 的现成库。

1.5 参数设置与实验设计

- K-means: 测试 $k = 2, 3, 4, 5$；随机种子固定以便复现。
- DBSCAN: 测试 $\varepsilon = \{0.3, 0.5, 0.8, 1.0\}$ 与 min_samples $= \{3, 5, 7\}$。
- 对每组参数记录：Accuracy（对齐簇标签后）、Silhouette、Calinski-Harabasz 及噪声点数（DBSCAN）。

1.6 评估指标设计与实现

本次实验采用三类指标评估聚类效果：

1) **Accuracy（准确率，需标签对齐）**：聚类是无监督的，簇标签与真实标签无对应关系。我们通过标签对齐（枚举映射或匈牙利算法）找到使匹配数量最大的映射 π，Accuracy 定义为

$$\text{Accuracy} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{\pi(\hat{y}_i) = y_i\}.$$

2) **轮廓系数（Silhouette）**：对每个样本 i，令 $a(i)$ 为该样本与同簇其他点的平均距离，$b(i)$ 为其到最近异簇点的平均距离，则样本的轮廓系数为

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1].$$

整体轮廓系数为所有样本 $s(i)$ 的平均值（噪声点在计算时可按 0 处理或忽略，本实现忽略标签为 -1 的噪声点）。该指标兼顾簇内紧密度与簇间分离度。

3) **Calinski-Harabasz 指数（CH）**：基于簇间和簇内方差比率定义，计算公式为

$$\text{CH} = \frac{\text{trace}(B)/(k-1)}{\text{trace}(W)/(n-k)} = \frac{\text{between}/(k-1)}{\text{within}/(n-k)},$$

其中 $\text{between} = \sum_j n_j ||\mu_j - \mu||^2$，$\text{within} = \sum_j \sum_{i \in C_j} ||x_i - \mu_j||^2$，$\mu$ 为样本总体均值，n_j 为簇 j 大小。CH 指数越大，说明簇间差异越显著、簇内越紧凑。

上述指标均在代码中手写实现，避免使用 sklearn 的现成度量函数。

1.7 实验结果

1.7.1 K-means 结果

下表给出 K-means 在不同 k 下的评估结果：

表 1.1 K-means 不同 k 下的评估结果

k	Accuracy	Silhouette	Calinski-Harabasz
2	0.6667	0.5818	251.3493
3	0.8533	0.4565	239.4845
4	0.6467	0.4151	206.2286
5	0.3733	0.3965	171.3978

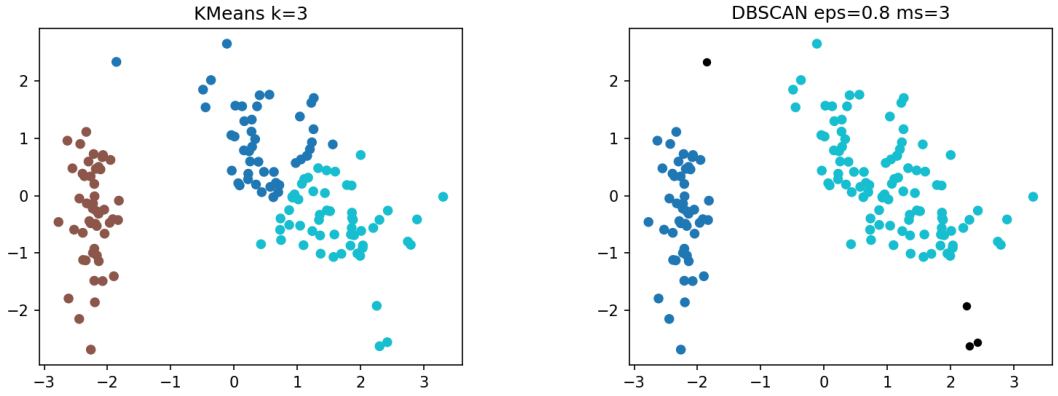
1.7.2 DBSCAN 结果

表中呈现 DBSCAN 多组参数结果，并给出噪声点数：

表 1.2 DBSCAN 参数组合结果

ε	min_samples	Accuracy	Silhouette	CH	Noise
0.3	3	0.12	0.3974	564.8942	84
0.3	5	0.12	0.6302	4169.2823	120
0.3	7	0.08	0.6276	11827.2833	129
0.5	3	0.5733	0.2591	90.4069	17
0.5	5	0.56	0.6559	458.5748	34
0.5	7	0.52	0.4360	261.2142	46
0.8	3	0.66	0.5979	285.6262	4
0.8	5	0.66	0.5979	285.6262	4
0.8	7	0.6467	0.6065	306.0209	7
1.0	3	0.66	0.5868	258.6397	1
1.0	5	0.66	0.5952	278.9375	3
1.0	7	0.66	0.5952	278.9375	3

1.7.3 可视化结果对比



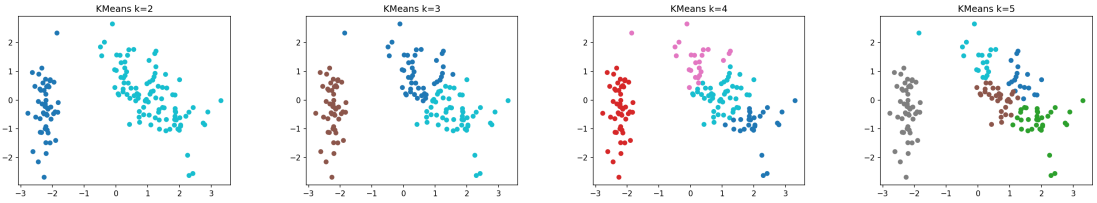
(a) K-means 聚类结果（$k = 3$）

(b) DBSCAN 聚类结果（$\varepsilon = 0.8$, min_samples = 3）

图 1.1 K-means 与 DBSCAN 聚类结果对比（PCA 2D 投影）

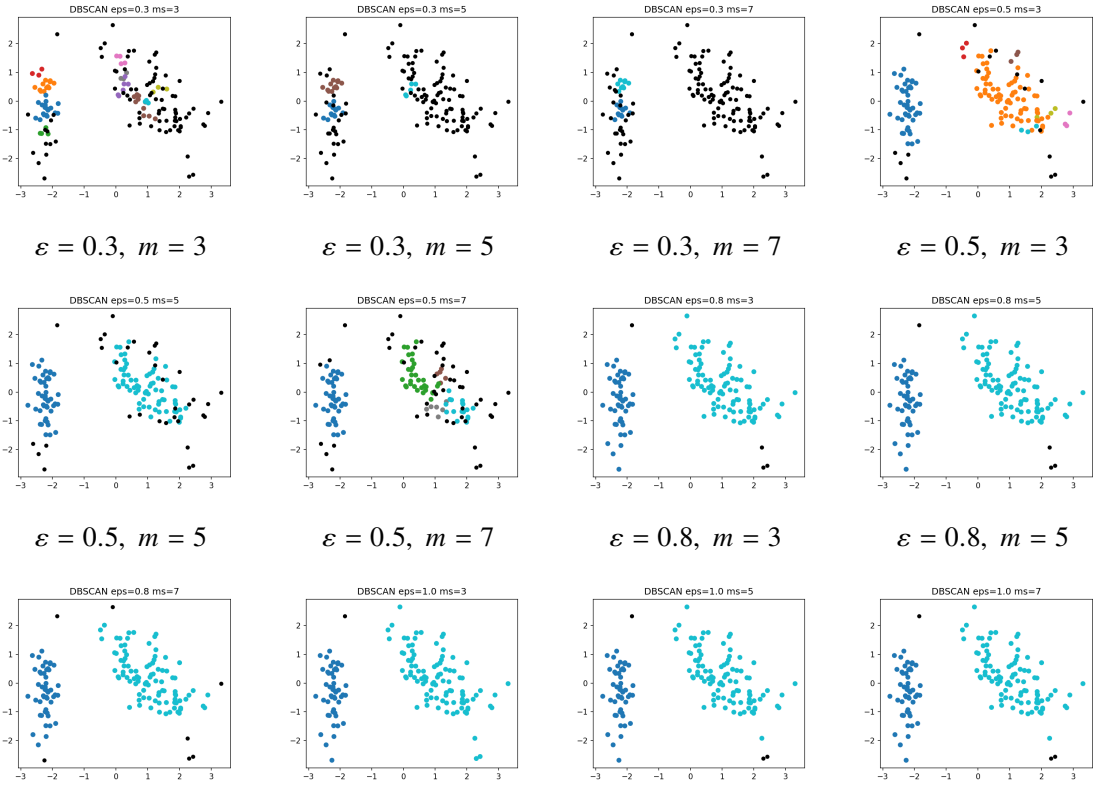
1.7.4 全部实验可视化一览

在此处将所有预先设置 K-means 参数的结果以及 DBSCAN 所有参数组合的结果全部展示出来。



K=2　　K=3　　K=4　　K=5

图 1.2 K-means 在不同 k 下的 PCA 可视化（四图并列）



$\varepsilon = 0.3,\ m = 3$　　$\varepsilon = 0.3,\ m = 5$　　$\varepsilon = 0.3,\ m = 7$　　$\varepsilon = 0.5,\ m = 3$

$\varepsilon = 0.5,\ m = 5$　　$\varepsilon = 0.5,\ m = 7$　　$\varepsilon = 0.8,\ m = 3$　　$\varepsilon = 0.8,\ m = 5$

$\varepsilon = 0.8,\ m = 7$　　$\varepsilon = 1.0,\ m = 3$　　$\varepsilon = 1.0,\ m = 5$　　$\varepsilon = 1.0,\ m = 7$

图 1.3 DBSCAN 在所有参数组合下的聚类可视化（PCA 2D，4 列 x 3 行）

1.8 实验结果分析与评价

1.8.1 实验结果分析

- 从表 1.1 可见，当 $k = 3$ 时 K-means 的 Accuracy 达到最高（0.8533），这与鸢尾花数据集本身的三类结构一致，说明 K-means 在簇为凸且大小相近时表现较好。
- 轮廓系数在 $k = 2$ 时最高（0.5818），这是因为把某两类合并后簇内紧密度提升，但 Accuracy 并非最优；因此不同指标反映聚类质量的侧重点不同，应同时考量。
- DBSCAN 在合适参数下（例如 $\varepsilon = 0.8$, min_samples $= 3$）能达到较高的 Accuracy（0.66）且噪声点少（4 个），说明其能够发现非球形簇与剔除离群点，但对参数较为敏感（见表 1.2）。
- Calinski-Harabasz 指数在某些 DBSCAN 参数下异常较大（例如 $\varepsilon = 0.3$ 且 min_samples 较大时），这通常由于大量点被标记为噪声或簇数极少导致簇间/簇内方差比例异常，应结合其它指标与可视化共同判断。

1.8.2 实验结果评价

本节结合实验设计与上表/可视化结果对两种算法的性能、适用性与参数敏感性给出评价与建议。

- **K-means 性能与适用性**：实验中 K-means 在 $k = 3$ 时取得最高 Accuracy（0.8533），说明在鸢尾花数据上簇数 $k = 3$ 与真实类别一致时模型能较好恢复类标。K-means 的 Silhouette 在 $k = 2$ 更高（0.5818），这表明在二维 PCA 投影上合并两类可获得更紧密的簇内结构，但从分类匹配角度并非最优。综上：当关注“恢复真实类别”时优先选择与先验一致的 k；当关注簇内部结构紧密度与可分性时可结合 Silhouette 指标选择 k。
- **DBSCAN 的表现与参数敏感性**：DBSCAN 在不同 ε 与 min_samples 下表现差异显著。较小的 ε（如 0.3）会将大量点标记为噪声，导致 CH 指数异常增大（由于有效簇数很少且簇间方差/簇内方差比率失衡），Accuracy 同样偏低；而 $\varepsilon = 0.8$ 且 min_samples = 3 时既能保持较少的噪声点（4 个），又能取得较高的 Silhouette 与合理的 Accuracy（0.66）。因此对 DBSCAN 建议通过参数扫描并结合可视化与噪声比例来选择参数，或使用 kNN 距离曲线（Elbow 法）辅助确定 ε。
- **指标冲突的理解与实用建议**：Accuracy、Silhouette 与 CH 指数各有侧重点：

  - Accuracy 评估聚类结果与真实标签的一致性，依赖于类别对齐；适合有监督参考时使用；
  - Silhouette 更关注簇内紧密度与簇间分离度，适合无监督质量判断；
  - CH 指数对簇数量及簇大小敏感，在存在大量噪声或极不平衡簇时可能出现异常值。

  因此在选择模型与参数时，推荐同时参考三者并结合 PCA 可视化结果，而非单一指标。

- **鲁棒性与复现性**：当前实验为一次性运行的结果，存在随机初始化与噪声导致波动的风险。实用建议：对 K-means 进行多次随机初始化（例如 10 次）并取最小 SSE/最大 CH 的结果；对 DBSCAN 在邻域参数附近做更细粒度扫描并统计噪声比例的稳定性。

2 总结

2.1 实验总结

本实验在鸢尾花（Iris）数据集上实现并对比了手写的 K-means 与 DBSCAN 两种聚类算法。主要结论如下：

1. K-means 在 $k = 3$ 时取得最佳分类一致性（Accuracy = 0.8533），在簇为凸且大小相近的场景表现良好；2. DBSCAN 在 $\varepsilon = 0.8$、min_samples = 3 时取得较为平衡的结果（Accuracy 约为 0.66，噪声点 4 个），但对参数敏感；3. 综合 Accuracy、Silhouette 与 Calinski–Harabasz 指数及可视化可知：K-means 适合球形、簇数已知或可近似估计的场景，DBSCAN 更适合发现任意形状簇并能自动识别离群点，但需细调参数。

基于以上结果分析，可以对 K-means 使用多次随机初始化并取最优解；对 DBSCAN 在参数附近进行更细粒度的扫描并结合 kNN 距离曲线辅助选取 ε。

2.2 技术学习总结

通过本次实验，我在以下几方面有明显收获：

- 深入理解并手写实现 K-means 与 DBSCAN 的核心流程与边界情形（如空簇处理、噪声点标识与簇扩展逻辑）；
- 掌握了聚类评估指标的含义与计算：Accuracy 的标签对齐方法、Silhouette 系数与 Calinski–Harabasz 指数的计算与适用场景；
- 强化了实验设计与复现性实践，包括参数网格搜索、随机种子控制、多次试验统计与结果可视化实践（PCA 投影）；

2.3 心得体会

本次作业强化了我对无监督学习方法原理与工程实现的理解。体会到参数敏感性是聚类方法的核心挑战之一，单一评价指标往往不足以全面反映聚类质量，因此需要结合多种指标和可视化结果进行判断。后续可在更多数据集上做稳定性测试，或尝试混合基于距离与密度的算法以提升鲁棒性。

参考文献

[1] 王昌栋. 第二章数据科学基础理论[Z]. 2026.

附录 A 实验代码

A.1 聚类算法代码清单

本次聚类实验的核心代码位于 code/ 目录下，主要文件如下：

- code/Kmeans.py：手写实现的 KMeans 类
- code/DBSCAN.py：手写实现的 DBSCAN 类
- code/metrics.py：手写实现的评估指标
- code/pca_viz.py：基于 SVD 的 PCA 实现以便可视化。
- code/main.py：主函数驱动脚本

A.2 K-means 代码实现细节

```python
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from sklearn.datasets import load_iris


def load_and_scale_iris(save_path_npy=None, save_path_csv=None):
    iris = load_iris()
    X = iris.data.astype(float)
    mu = X.mean(axis=0)
    sigma = X.std(axis=0, ddof=0)
    Xs = (X - mu) / sigma
    y = iris.target
    if save_path_npy:
        np.save(save_path_npy, Xs)
    if save_path_csv:
        df = pd.DataFrame(Xs, columns=iris.feature_names)
        df['target'] = y
        df.to_csv(save_path_csv, index=False)
    return Xs, y


```

```python
class KMeans:
    def __init__(self, n_clusters=3, max_iters=300, tol=1e-4, random_state=None):
        self.n_clusters = n_clusters
        self.max_iters = max_iters
        self.tol = tol
        self.random_state = random_state

    def fit(self, X):
        rng = np.random.default_rng(self.random_state)
        n_samples, n_features = X.shape
        # init centroids as random samples
        idx = rng.choice(n_samples, size=self.n_clusters, replace=False)
        centroids = X[idx].astype(float)

        for _ in range(self.max_iters):
            dists = cdist(X, centroids, metric='euclidean')
            labels = np.argmin(dists, axis=1)
            new_centroids = np.zeros_like(centroids)
            for k in range(self.n_clusters):
                members = X[labels == k]
                if len(members) == 0:
                    # reinit empty cluster to a random point
                    new_centroids[k] = X[rng.integers(0, n_samples)]
                else:
                    new_centroids[k] = members.mean(axis=0)
            shift = np.linalg.norm(new_centroids - centroids)
            centroids = new_centroids
            if shift <= self.tol:
                break

        self.cluster_centers_ = centroids
        self.labels_ = labels
        return self

    def predict(self, X):
        dists = cdist(X, self.cluster_centers_, metric='euclidean')
        return np.argmin(dists, axis=1)
```

```
60
61      def fit_predict(self, X):
62          self.fit(X)
63          return self.labels_
64
65
66  if __name__ == '__main__':
67      Xs, y = load_and_scale_iris(save_path_npy='data/iris_scaled.npy',
        save_path_csv='data/iris_scaled.csv')
68      print('Saved scaled iris to data/iris_scaled.* (shape {})'.format(Xs.
        shape))
```

A.3 DBSCAN 代码实现细节

```
1   import numpy as np
2   from scipy.spatial import distance
3
4
5   class DBSCAN:
6       def __init__(self, eps=0.5, min_samples=5):
7           self.eps = eps
8           self.min_samples = min_samples
9
10      def fit_predict(self, X):
11          X = np.asarray(X)
12          n = X.shape[0]
13          labels = np.full(n, -2, dtype=int)  # -2 unvisited, -1 noise
14          cluster_id = 0
15          # precompute distance matrix (small data set)
16
17          for i in range(n):
18              if labels[i] != -2:
19                  continue
20              # neighbors including itself
21              if len(neigh) < self.min_samples:
22                  labels[i] = -1  # noise
23              else:
```

```
                # create new cluster
                seeds = [p for p in neigh if p != i]
                j = 0
                while j < len(seeds):
                    point = seeds[j]
                    if labels[point] == -1:
                        labels[point] = cluster_id
                    if labels[point] == -2:
                        labels[point] = cluster_id
                        # expand
                        if len(point_neigh) >= self.min_samples:
                            for p in point_neigh:
                                if p not in seeds:
                                    seeds.append(p)
                    j += 1
                cluster_id += 1

        self.labels_ = labels
        self.n_clusters_ = len([c for c in set(labels) if c >= 0])
        return labels


if __name__ == '__main__':
    from sklearn.datasets import load_iris
    iris = load_iris()
    X = iris.data
    model = DBSCAN(eps=0.7, min_samples=3)
    labels = model.fit_predict(X)
    print('Found clusters:', set(labels))
```